

Session 2: AI for Coding – Practical Workflows and Demos

Elira Kuka and Tanner Regan

The George Washington University

Plan for Today: hands-on workflows

1. Demos with Stata:

- Replication package *EK*
- Checking RA work *EK*
- Quick demo of with VS Code *TR*

2. Python and Claude Code

- Python in VS Code overview *TR*
 - ▶ libraries and environments
 - ▶ Jupyter extension
- Data analysis demo with Python
 - ▶ Python example with Claude Code from VS Code

Coding with Claude Code

Demo: Replication Package

- ▶ Task: take a paper written in Latex and code folder, and produce repl package
- ▶ Steps we will walk through:
 1. Point Claude at the project folder
 2. Tell Claude to follow AEA instructions for replication package
 3. Tell Claude to test all code runs and produces correct output
 4. Save steps for future work (save as skill)
- ▶ What to watch for:
 - Bad prompting (mine!)
 - More inefficient than I hoped, though MUCH faster than myself

Demo: Check RA work

- ▶ Task: understand RA work and check for issues
 1. Ask Claude to write a plan before touching any files
 2. Review the plan in Markdown; request changes to the approach
 3. Approve; Claude executes and produces a clean, commented script
 4. Compare output to Ask-mode version: same result, better code
- ▶ Reviewing session history: how to audit what Claude did
- ▶ Ask Claude to update README with what was done (good practice)

How to Verify AI Code

- ▶ AI code can look completely correct and be wrong in subtle ways
- ▶ For replication package, easy to check: results either replicates or not.
- ▶ Verification checklist for data work;
 - Does the code run without errors? (necessary, not sufficient)
 - Do merged/unmerged counts make sense at each merge step?
 - I ask my RA to do this
 - I am sure can ask Claude to provide these numbers
 - Do obdservation counts and means match codebook or prior literature?
 - Can you reproduce a known result (e.g., a published Table 1)?
 - Ask Claude: “what assumptions did you make?” — it will often reveal them

Stata in VS Code demo

- ▶ Very quickly - how to run stata from VS Code
 - ▶ Use the [Stata Workbench Extension](#) (setup guide at link)
 - ▶ Make sure the extension can 'find' stata
 - ▶ Needs to connect to python to run 'uv.exe'
 - ▶ Also had trouble reading from outside of it's current working directory
 - ▶ Claude wrote up [instructions](#) to fix both
 - ▶ Work (mostly) as normal
 - ▶ Some things different, e.g. 'browse' (needs point and click), 'desc' and 'count' (sometimes don't run from script)
 - ▶ Otherwise, Claude Code can work with Stata similarly to previous example

▷ *Stata & VS Code Demo*

Python in VS Code overview

Why Python?

► Claes Backman

Python for Economists: Common Tasks

- Data cleaning with `pandas`: merging, collapsing, reshaping
- Data collection: `requests`, `BeautifulSoup` for scraping (Session 3)
- Figures: `matplotlib`, `seaborn` — publication-quality plots
- Applied econometrics: `statsmodels`, `linearmodels` (panel data, IV)
- Anything else you might want a computer to do!
- Claude Code can translate your Stata code to Python (or vice versa):
 - Running analyses you cannot do in Stata
 - Working with large datasets where Python is faster

Python basics

- ▶ Libraries and Environments
 - ▶ Anaconda manages Python installations and packages
 - ▶ Each project gets its own **environment**: a fixed set of packages and versions
 - Prevents conflicts between projects
 - Ensures others can reproduce your results exactly (store in `.yaml` file)
 - ▶ Ask Claude Code to create and activate an environment for you
- ▶ Three ways to run Python code
 - **Script** (`.py`): run the full pipeline — best for production code
 - **Interactive window**: run `.py` scripts cell-by-cell — best of both worlds
 - **Jupyter notebook** (`.ipynb`): interactive cells, inline output — good for exploration

Data Analysis Demo with Python

I AM IMPRESSED

I AM IMPRESSED

- ▶ This example took about 2.5 days of my work
- ▶ Before Claude Code:
 - ▶ 2-4 weeks of my time
 - ▶ 1-2 months of very good PhD RA time
 - ▶ 2+ months of decent MS RA time
- ▶ Claude solution still more efficient and elegant
- ▶ More discussion later...

Python Exercise

Produced data (sampling frame for survey) and a [set of slides](#) in four steps:

1. Download data from Kigali City
2. Download data from Alpha Earth
3. Build a sample frame with visualization
4. Test an event study design with google satellite embeddings

Python Exercise

Produced data (sampling frame for survey) and a [set of slides](#) in four steps:

1. Download data from Kigali City
 2. Download data from Alpha Earth
 3. Build a sample frame with visualization
 4. Test an event study design with google satellite embeddings
-
- ▶ Paul GP has 3 complementary blog posts w/ youtube lectures
 - ▶ [Data Analysis](#), [Web Scraping](#), [Large Datasets](#)

Download data from Kigali City

1. Wrote [Prompt data scraping for Kigali.md](#) pasted into “plan mode”
 - ▶ I had to give permission on a few steps, e.g. searching URLs.
2. Claude generated the plan [kigali-city-data-mutable-frost.md](#) (~10 mins)
3. Executed the plan which created the script [download_kigali_gis_v0.py](#)
4. Ran the code without edits (using Jupyter window). Two issues:
 - ▶ Claude added packages to the yml but didn't update the conda environment. Chatted with Claude to identify and fix this.
 - ▶ I had daycare pickup and the code was still running. Claude adjusted the code so it would restart where I left off, rather than restarting completely.
5. Ran the updated script [download_kigali_gis.py](#) at home → success!
 - ▶ VS Code diff: right-click script.py “Select for Compare”, right-click script2.py “Compare with Selected”. Gives a clean side-by-side diff in the editor.

Download data from Google Earth Engine

1. Wrote [Prompt-data-scraping-Alpha-Earth.md](#) pasted into “plan mode”
2. Claude generated the plan [alpha-earth-data-witty-planet.md](#) (~10 mins)
3. Executed the plan to create [download_satellite_embedding_v0.py](#)
4. Ran the code without edits (using Jupyter window). issues:
 - ▶ Needed to add my GEE project code (ee-tannerregan-gwu)
 - ▶ More substantial errors appeared, I iterated with Claude to make [*_v1.py](#)
5. Ran the updated script successfully
6. Asked Claude to revise the code now that we had iterated
 - ▶ Simplify code lines from the debugging/iterating process and make sure well commented [download_satellite_embedding.py](#)

Download data from Google Earth Engine

1. Wrote [Prompt-data-scraping-Alpha-Earth.md](#) pasted into “plan mode”
2. Claude generated the plan [alpha-earth-data-witty-planet.md](#) (~10 mins)
3. Executed the plan to create [download_satellite_embedding_v0.py](#)
4. Ran the code without edits (using Jupyter window). issues:
 - ▶ Needed to add my GEE project code (ee-tannerregan-gwu)
 - ▶ More substantial errors appeared, I iterated with Claude to make [*_v1.py](#)
5. Ran the updated script successfully
6. Asked Claude to revise the code now that we had iterated
 - ▶ Simplify code lines from the debugging/iterating process and make sure well commented [download_satellite_embedding.py](#)

Check out VS Code Diff again!

Build a sample frame with visualization

- ▶ Wrote out three prompts (took me longer to get started with this exercise)
 - ▶ [Prompt-prepare-sample-frame.md](#) (writes draft)
 - ▶ wrote plan: [make-a-plan-based-zippy-sprout.md](#)
 - ▶ [Prompt-round2-prepare-sample-frame.md](#) (adjustments)
 - ▶ lost track of this plan (sorry)
 - ▶ [Prompt-round3-prepare-sample-frame.md](#) (added one new step)
 - ▶ wrote plan: [create-a-plan-that-witty-cosmos.md](#)
- ▶ The final python script is [prepare_sample_frame.py](#)

Test an event study with satellite embeds

- ▶ Wrote a prompt to run a couple example designs
 - ▶ [Prompt-event-study.md](#) was my initial prompt
 - ▶ This time I did many iterations and didn't keep track (sorry)
- ▶ The final python script was [event_study.py](#)
 - ▶ Claude also make the [4_slides_event_study.tex](#) that built the slides at the top of this section

Implications for RAs and research students

- ▶ My willingness to pay for PhD and MS student RAs is down
- ▶ Conditional on working with an RA, task allocation will change:
 - ▶ ↓ More menial tasks like undergrad RA (data collection, subjective classification, etc.)
 - ▶ ↑ More high level conceptual tasks (design survey questionnaires, design data cleaning / empirical strategy)
- ▶ This has implications for students
 - ▶ They still need to know the fundamentals of coding to make CC productive, but training opportunities will become more scarce
 - ▶ Otherwise, RAships that focus more on high level conceptual tasks will probably help make them better researchers. But IMO requires higher RA ability and trust

Wrapping Up

Key Takeaways from Sessions 1 & 2

1. AI handles the coding mechanics — your job is to verify the output and drive the analysis
2. Use Plan mode for any non-trivial task; it gives you an audit trail and catches mistakes early
3. Git + Claude Code = version-controlled, reproducible research without learning Git commands
4. CLAUDE.md files carry your preferences across sessions so you do not repeat yourself
5. Replication packages are now a hours-long task, not a weeks-long one
6. The critical skill is *verification*: check outputs, not just that the code runs

Things to Try Before the Next Session

- ▶ Set up VS Code with Claude Code, Git, and Python (use the setup guide)
- ▶ Ask Claude to write a small data task in your preferred language — verify the output
- ▶ Initialize a Git repo for one of your current projects
- ▶ Write a CLAUDE.md file for that project
- ▶ Coming up in Session 3: Finding and Collecting Data
 - Identifying datasets, scraping, APIs, parsing PDFs at scale

Questions?

- ▶ Questions on anything from Sessions 1 or 2?
- ▶ What tasks in your own research would you most like to try this on?